

Fonctionnement du jeu Alterdune

Ce document résume le fonctionnement du jeu, les principales classes, les méthodes importantes, les règles de jeu et les choix d'architecture.

Principe général

Alterdune est un jeu console de type RPG au tour par tour. Le joueur affronte des monstres choisis aléatoirement depuis `data/monsters.csv`. Un combat peut être gagné de deux façons :

- tuer le monstre avec `FIGHT` ;
- remplir sa jauge `Mercy`, puis utiliser `MERCY` pour l'épargner.

La partie se termine lorsque le joueur atteint 10 victoires. La fin dépend de l'historique des combats :

- fin pacifiste : aucun monstre tué ;
- fin génocidaire : aucun monstre épargné ;
- fin neutre : mélange de monstres tués et épargnés.

Démarrage du jeu

Le point d'entrée est `src/main.cpp`.

Au lancement :

1. Le joueur saisit le nom de son personnage.
2. Un `Player` est créé avec 30 HP, 5 ATK et 2 DEF.
3. Le catalogue d'actions ACT est rempli dans le code.
4. Les items sont chargés depuis `data/items.csv`.
5. Les monstres sont chargés depuis `data/monsters.csv`.
6. Le jeu affiche un résumé de session puis lance le menu principal.

Les fichiers CSV sont chargés avec `CsvLoader`. Le programme tente aussi des chemins de secours (`../data/...`) afin de fonctionner depuis le dossier racine ou depuis un dossier de build.

Menu principal

La boucle principale est gérée par `Game::mainMenu()`.

Le joueur peut choisir :

- `Bestiaire` : affiche les monstres déjà vaincus ;
- `Démarrer un combat` : lance un combat contre un monstre aléatoire ;
- `Statistiques` : affiche HP, victoires, monstres tués et épargnés ;
- `Items` : affiche l'inventaire et permet d'utiliser un item hors combat ;
- `Quitter` : arrête la partie.

`Game::run()` appelle simplement `mainMenu()`.

Règles de combat

Le combat est géré par la classe `Combat`.

Chaque tour affiche :

- les HP du joueur ;
- les HP du monstre ;
- la jauge `Mercy` du monstre ;
- les derniers événements du combat ;
- le menu `FIGHT`, `ACT`, `ITEM`, `MERCY`.

FIGHT

`Combat::playerFight()` calcule des dégâts avec `RandomService::rollDamage()`, puis appelle `Monster::takeDamage()`.

Si le monstre tombe à 0 HP, le combat est gagné avec le résultat `KILLED`.

ACT

`Combat::playerAct()` applique une action du catalogue `ActCatalog`.

Chaque `ActAction` contient :

- un identifiant (`JOKE`, `COMPLIMENT`, `INSULT`, etc.) ;
- un texte affiché dans le journal de combat ;
- une variation de `Mercy`.

Les actions positives augmentent la jauge `Mercy`. Les actions négatives comme `INSULT` et `THREATEN` la diminuent. La valeur reste toujours bornée entre 0 et l'objectif du monstre grâce à `Monster::setMercy()`.

ITEM

`Combat::playerItem()` utilise un objet de l'inventaire. Les items actuels sont uniquement de type `HEAL`. Ils soignent le joueur puis sont consommés.

La logique est portée par :

- `Inventory::use()` : vérifie la quantité, applique l'item et décrémente ;
- `Item::apply()` : applique l'effet sur le joueur ;
- `Player::heal()` : restaure les HP sans dépasser le maximum.

MERCY

`Combat::playerMercy()` vérifie si la jauge `Mercy` du monstre a atteint son objectif. Si oui, le monstre est épargné et le combat est gagné avec le résultat `SPARED`.

Si la jauge est insuffisante, la tentative échoue et le monstre joue son tour.

Tour du monstre

Après une action valide du joueur, si le monstre est encore vivant, `Combat::monsterTurn()` lui fait attaquer le joueur.

Si le joueur tombe à 0 HP, le combat renvoie `PLAYER_DEFEATED` et la partie est perdue immédiatement.

Choix différent sur les dégâts

Le sujet demande une formule basée sur un tirage aléatoire entre 0 et les HP maximum du défenseur :

```
dégâts = rand(0, HP_max_défenseur)
```

Cette formule autorise donc les coups ratés (`dégâts = 0`) et n'utilise pas les statistiques `ATK` et `DEF`.

Dans ce projet, le choix est différent. Les dégâts sont calculés dans `RandomService::rollDamage(int atk, int def)` :

```
base = atk + rand(0, atk / 2)
réduction = floor(def * 0.2)
dégâts = max(1, base - réduction)
```

Conséquences de ce choix :

- l'attaque (`ATK`) influence directement les dégâts ;
- la défense (`DEF`) réduit les dégâts reçus ;
- il n'y a pas de coup totalement raté, car les dégâts minimum valent 1 ;
- les combats sont plus prévisibles et mieux liés aux statistiques des entités ;
- les champs `ATK` et `DEF` du CSV ont un vrai impact gameplay.

Ce choix s'écarte donc du sujet sur la formule exacte, mais il renforce le rôle des statistiques déjà présentes dans `monsters.csv`.

Objets et classes principales

Entity

`Entity` est la classe de base pour les entités qui ont des statistiques de combat.

Attributs principaux :

- nom ;
- HP actuels ;
- HP maximum ;
- attaque ;
- défense.

Méthodes importantes :

- `getHp()`, `getMaxHp()`, `getAtk()`, `getDef()` : accesseurs ;

- `isAlive()` : indique si l'entité a encore des HP ;
- `takeDamage()` : retire des HP ;
- `setHp()` : borne les HP entre 0 et le maximum.

Player

`Player` hérite de `Entity`.

Il ajoute :

- le nombre de monstres tués ;
- le nombre de monstres épargnés ;
- le nombre de victoires ;
- un `Inventory`.

Méthodes importantes :

- `addKill()`, `addSpare()`, `addVictory()` ;
- `heal()` ;
- `getInventory()`.

Monster

`Monster` hérite de `Entity` et représente la base commune des ennemis.

Il ajoute :

- une catégorie (`NORMAL`, `MINIBOSS`, `BOSS`) ;
- une jauge `Mercy` ;
- un objectif de `Mercy` ;
- une liste d'identifiants ACT disponibles.

Méthodes importantes :

- `getAllowedActCount()` : méthode polymorphe ;
- `clone()` : permet de dupliquer un monstre avant un combat ;
- `setMercy()` : applique le bornage de la jauge.

NormalMonster, MiniBossMonster, BossMonster

Ces classes spécialisées héritent de `Monster`.

Elles définissent le nombre d'actions ACT disponibles :

- `NormalMonster` : 2 actions ;
- `MiniBossMonster` : 3 actions ;
- `BossMonster` : 4 actions.

C'est le principal usage du polymorphisme dans le projet.

ActAction et ActCatalog

Act représente une action ACT individuelle.

ActCatalog stocke toutes les actions disponibles dans une **map**, indexée par identifiant.

Méthodes importantes :

- **add()** : ajoute une action ;
- **has()** : vérifie si une action existe ;
- **get()** : récupère une action ou lance une erreur si elle est inconnue ;
- **list()** : retourne toutes les actions.

Item et Inventory

Item représente un objet utilisable. Actuellement, seul le type **HEAL** est implémenté.

Inventory stocke :

- les quantités par nom d'item ;
- les définitions complètes des items.

Cette séparation permet de connaître à la fois combien d'items restent et quel effet appliquer.

Bestiary et BestiaryEntry

Le bestiaire garde l'historique des combats gagnés.

Chaque **BestiaryEntry** contient :

- le nom du monstre ;
- sa catégorie ;
- ses statistiques ;
- le résultat du combat (**KILLED** ou **SPARED**).

Game

Game coordonne la partie.

Il possède :

- le joueur ;
- le bestiaire ;
- le catalogue ACT ;
- la liste des monstres ;
- l'interface console ;
- l'état de fin de partie.

Méthodes importantes :

- **mainMenu()** : boucle principale ;
- **startCombat()** : sélectionne un monstre aléatoire et lance un **Combat** ;
- **showStats()** ;
- **showItems()** ;

- `showBestiary()` ;
- `checkEndings()`.

Combat

Combat gère toute la logique d'un affrontement.

Il référence :

- le joueur ;
- le monstre courant ;
- le catalogue ACT ;
- l'interface terminal ;
- le service aléatoire ;
- un journal d'événements.

Méthodes importantes :

- `start()` : boucle principale du combat ;
- `playerFight()` ;
- `playerAct()` ;
- `playerItem()` ;
- `playerMercy()` ;
- `monsterTurn()`.

CsvLoader

CsvLoader charge les données depuis les fichiers CSV.

Il valide :

- le nombre de colonnes ;
- les entiers ;
- les types d'items ;
- les catégories de monstres ;
- les identifiants ACT ;
- le bon nombre d'actions selon la catégorie du monstre.

TerminalUI

TerminalUI centralise l'affichage console :

- bannières ;
- menus ;
- messages ;
- barres de progression ;
- tableaux ;
- lecture des choix utilisateur.

Cette classe évite de mélanger toute la logique d'affichage avec les classes de jeu.

Architecture

Le projet est organisé par responsabilités :

```
include/  
  acts/      actions ACT et catalogue  
  bestiary/  historique des monstres vaincus  
  entities/  joueur, monstres et classe de base Entity  
  io/        chargement CSV  
  items/     items et inventaire  
  systems/   orchestration du jeu et des combats  
  ui/        interface terminal  
  utils/     types communs et aléatoire  
  
src/  
  même découpage que include/, avec les implémentations .cpp  
  
data/  
  items.csv  
  monsters.csv
```

Les choix d'architecture importants sont :

- séparation `.hpp` / `.cpp` pour clarifier interfaces et implémentations ;
- encapsulation des attributs avec accesseurs ;
- héritage entre `Entity`, `Player` et `Monster` ;
- classes dérivées pour les catégories de monstres ;
- polymorphisme via `getAllowedActCount()` et `clone()` ;
- composition avec `Game`, `Inventory`, `Bestiary` et `ActCatalog` ;
- données modifiables via CSV sans recompiler le jeu ;
- logique d'affichage isolée dans `TerminalUI` ;
- logique aléatoire isolée dans `RandomService`.

Données CSV

items.csv

Format :

```
nom;type;valeur;quantité
```

Exemple :

```
Potion;HEAL;12;4
```

Le type **HEAL** soigne le joueur de **valeur** HP. La quantité indique combien d'exemplaires sont disponibles au début.

monsters.csv

Format :

```
catégorie;nom;hp;atk;def;mercyGoal;act1;act2;act3;act4
```

Règles :

- **NORMAL** doit avoir 2 actions ;
- **MINIBOSS** doit avoir 3 actions ;
- **BOSS** doit avoir 4 actions ;
- les emplacements inutilisés doivent valoir - ;
- les identifiants ACT doivent exister dans **ActCatalog**.

Résumé des règles importantes

- Une action valide du joueur consomme un tour.
- Après le tour du joueur, le monstre attaque s'il est encore vivant.
- Les HP sont toujours bornés entre 0 et les HP maximum.
- La jauge **Mercy** est toujours bornée entre 0 et son objectif.
- Un combat gagné ajoute une entrée au bestiaire.
- Une défaite contre un monstre termine la partie immédiatement.
- La partie se termine normalement après 10 victoires.